In [ ]:

Complete NUMPY Documentaion

In [23]:
```
import numpy as np
```

In [24]:
```
import pandas as pd
import matplotlib.pyplot as plt
```

In [21]:
```
from PIL import Image
```

In [22]:
```
img = Image.open(r'E:\Numpy\images.jpg')
img
```

Out[22]:

Array Creation functions

In [25]:
```
import numpy as np
```

In [26]:
```
# create an array from a list
a = np.array([1, 2, 3])
print("Array a:",a)
```

```
Array a: [1 2 3]
```

In [28]:
```
# Create an array with evenly spaced values
b = np. arange(0, 10, 3)    # Values from 0 to 10 with step 2
print("Array b:",b)
```

```
Array b: [0 3 6 9]
```

In [29]:
```
# Creaate an array filled with zeros
d= np.zeros((2,3))     # 2*3 array of zeros
print("Array d:\n",d)
```

```
Array d:
 [[0. 0. 0.]
 [0. 0. 0.]]
```

In [ ]:
```
# Create an array filled with zeros
d = np.zeros((2,3),dtype=int)  # 3x2 array of ones
print("Array e:\n",d)
```

```
Array e:
 [[0 0 0]
 [0 0 0]]
```

In [ ]:
```
# Create an array filled with ones
e = np.ones((3,2))    # 3x2 array of ones
print("Array e:\n", e)
```

```
Array e:
 [[1. 1.]
 [1. 1.]
 [1. 1.]]
```

In [33]:
```
# Create an identify matrix
f = np.eye(4)      #  4x4  identify matrix
print("Identify matrix f:\n",f)
```

```
Identify matrix f:
 [[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

In [ ]:
```

```

2. Array Manipulation Functions

In [34]:
```
# Reshaoe an array
a1 = np.array([1, 2, 3])
reshaped = np.reshape(a1, (3,1))   # Reshape to 1x3
print("Reshaped array:",reshaped)
```

```
Reshaped array: [[1]
 [2]
 [3]]
```

In [38]:
```
# Flatten an array
f1 = np.array([[1,2],  [3,4], [5,6], [7,8], [9,10]])
flattened = np.ravel(f1)    # Flatten to 1D array
print("flattened array:", flattened)
```

```
flattened array: [ 1  2  3  4  5  6  7  8  9 10]
```

In [39]:
```
# Transpose an array
e1 = np.array([[1,2],[3,4]])
transposed = np.transpose(e1)   #  Trancepose the array
print("Transposed array:\n",transposed)
```

```
Transposed array:
 [[1 3]
 [2 4]]
```

In [40]:
```
# Stack arrays vertically
a2 = np.array([1,2])
b2 = np.array([3,4])
stacked = np.vstack([a2,b2])   # stack a and b vertically
print("Stackd arrays:\n", stacked)
```

```
Stackd arrays:
 [[1 2]
 [3 4]]
```

In [42]:
```
# Stacked arrays horizontal
a2 = np.array([1,2])
b2 = np.array([3,4])
stacked = np.hstack([a2, b2])   # Stack a and b vertically
print("stacked arrays:\n",stacked)
```

```
stacked arrays:
 [1 2 3 4]
```

3. Mathematical Functions

In [44]:
```
# Add two arrays
g = np.array([1, 2, 3, 4,])
added = np.add(g,2)   # Add 2 to each element
print("Added 2 to g:",added)
```

```
Added 2 to g: [3 4 5 6]
```

In [45]:
```
# Squre each element
squred = np.power(g,2)   # Squre each element
print("Squared g:", squred)
```

```
Squared g: [ 1  4  9 16]
```

In [46]:
```
# Squred root of each element
sqrt_val = np.sqrt(g)   # Squre root of each element
print("Squre root of g:",sqrt_val)
```

```
Squre root of g: [1.         1.41421356 1.73205081 2.        ]
```

In [47]:
```
print(a)
print(g)
```

```
[1 2 3]
[1 2 3 4]
```

In [48]:
```
a3 = np.array([1, 2, 3])
dot_product = np.dot(a,g)    # Dot product of a and g
print("Dot product of a1 and a:", dot_product)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[48], line 2
      1 a3 = np.array([1, 2, 3])
----> 2 dot_product = np.dot(a,g)    # Dot product of a and g
      3 print("Dot product of a1 and a:", dot_product)

ValueError: shapes (3,) and (4,) not aligned: 3 (dim 0) != 4 (dim 0)
```

In [49]:
```
print(a)
print(a1)
```

```
[1 2 3]
[1 2 3]
```

In [50]:
```
a3 = np.array([1, 2, 3])
dot_product = np.dot(a1,a)   # Dot product of a and g
```

```
print("Dot product of a1 and a:",dot_product)
```

```
Dot product of a1 and a: 14
```

4. Statistical functions

In [52]:
```
s = np.array([1, 2, 3, 4])
mean = np.mean(s)
print("Mean of s:",mean)
```

```
Mean of s: 2.5
```

In [53]:
```
# Standard deviation of an array
var = np.var(s)
print("Variance of s:",var)
```

```
Variance of s: 1.25
```

In [54]:
```
# Standard deviation of an array
std_dev = np.std(s)
print("Standard deviation of s:",std_dev)
```

```
Standard deviation of s: 1.118033988749895
```

In [55]:
```
s
```

Out[55]:
```
array([1, 2, 3, 4])
```

In [57]:
```
# Minimum element of an array
minimum = np.min(s)
print("Min of s:",minimum)
```

```
Min of s: 1
```

In [58]:
```
# Maximum element of an array
maximum = np.max(s)
print("Max of s:",maximum)
```

```
Max of s: 4
```

5.Linear Algebra Functions

In [59]:
```
# Create a matrix
matrix = np.array([[1, 2], [3, 4]])
matrix
```

Out[59]:
```
array([[1, 2],
       [3, 4]])
```

6. Random Sampling functions

In [60]:
```
# Generate random values between 0 and 1
random_vals = np.random.rand(3)  # Array of 3 random values between 0 and 1
print("Random values:", random_vals)
```

```
Random values: [0.28796171 0.30540356 0.79425648]
```

In [61]:
```
# Set seed for reproducibility
np.random.seed(0)

# Generate random values between 0 and 1
random_vals = np.random.rand(3)  # Array of 3 random values between 0 and 1
print("Random values:", random_vals)
```

```
Random values: [0.5488135  0.71518937 0.60276338]
```

In [62]:
```
# Set seed for reproducibility
np.random.seed(0)

# Generate random integers
rand_ints = np.random.randint(0, 10, size=5)  # Random integers between 0 and 10
print("Random integers:", rand_ints)
```

```
Random integers: [5 0 3 3 7]
```

7. Boolean & Logical Fuctions

In [63]:
```
# Check if all elements are True

# all
logical_test = np.array([True, False, True])
all_true = np.all(logical_test)  # Check if all are True
print("All elements True:", all_true)
```

```
All elements True: False
```

In [64]:
```
# Check if all elements are True
logical_test = np.array([True, False, True])
all_true = np.all(logical_test)  # Check if all are True
print("All elements True:", all_true)
```

```
All elements True: False
```

In [65]:
```
# Check if all elements are True
logical_test = np.array([False, False, False])
all_true = np.all(logical_test)  # Check if all are True
print("All elements True:", all_true)
```

```
All elements True: False
```

In [66]:
```
# Check if all elements are True
logical_test = np.array([False, True, False])
all_true = np.all(logical_test)  # Check if all are True
print("All elements True:", all_true)
```

```
All elements True: False
```

In [67]:
```
# Check if any elements are True
# any
any_true = np.any(logical_test)  # Check if any are True
print("Any elements True:", any_true)
```

```
Any elements True: True
```

8. Set Operations

In [70]:
```
# Intersection of two arrays
set_a = np.array([1, 2, 3, 4])
set_b = np.array([3, 4, 5, 6])
intersection = np.intersect1d(set_a, set_b)
print("Intersection of a and b:", intersection)
```

```
Intersection of a and b: [3 4]
```

In [71]:
```
# Union of two arrays
union = np.union1d(set_a,set_b)
print("Union of a and b;", union)
```

```
Union of a and b; [1 2 3 4 5 6]
```

9. Array Attribute Fuctions

In [72]:
```
# Array attributes
a = np.array([1, 2, 3])
shape = a.shape  # Shape of the array
size = a.size    # Number of elements
dimensions = a.ndim  # Number of dimensions
dtype = a.dtype   # Data type of the array

print("Shape of a:", shape)
print("Size of a:", size)
print("Number of dimensions of a:", dimensions)
print("Data type of a:", dtype)
```

```
Shape of a: (3,)
Size of a: 3
Number of dimensions of a: 1
Data type of a: int64
```

10. Other Fuctions

In [73]:
```
# Crete a copy of an array
a = np.array([1, 2, 3])
copied_array = np.copy(a)   #  Create a copy of array a
print("Copied array;", copied_array)
```

```
Copied array; [1 2 3]
```

In [74]:
```
# Size in bytes of an array
array_size_in_bytes = a.nbytes   # Size in bytes
print("Size of a in byets:",array_size_in_bytes)
```

```
Size of a in byets: 24
```

In [75]:
```
# Check if two arrays share memory
shared = np.shares_memory(a, copied_array)  # Check if arrays share memory
print("Do a and copied_array share memory?", shared)
```

```
Do a and copied_array share memory? False
```

=== THANK YOU ===

In [ ]: